Fine-Tuning, Visual Grounding and Test-Time Compute in Small Open Models

Zarman Ali Awan
28100092
28100092@lums.edu.pk

Abstract

We study three ways of adapting or spending compute on small open models, all on 16 GB T4 GPUs. *Fine-tuning:* training Qwen3-0.6B on 40k MetaMathQA examples raises GSM8K from 43.6% to 59.7–65.0% for all five methods, but only full fine-tuning loses reading comprehension (BoolQ -4.0 pp); an L2-SP penalty keeps the full math gain while shrinking parameter displacement by 41% and removing the BoolQ loss, and orthogonally protected LoRA (OPLoRA) behaves like plain LoRA, whose updates already avoided the protected subspace. Manual inspection shows math-style "The answer is:" lines and repetition loops on unrelated prompts that benchmark scores do not reveal. *Visual grounding:* a SigLIP–Qwen3 model trained with a three-stage curriculum answers 67.2% of annotation-derived grounding questions and 55.2% of natural VQAv2 questions with the correct image, versus 14.1% and 26.8% with the complementary image; image sensitivity appears only once paired questions are trained, and natural questions remain more prior-driven. Pooling to 49 visual tokens keeps accuracy within the intervals at half the training time. *Test-time compute:* with 32 rollouts, oracle coverage exceeds 90% on MATH and MathVista, yet self-consistency reaches only 73.8% and 63.9%; quality-weighted voting adds at most 1.8 pp, highest-score selection hurts, and agreement-based adaptive stopping matches fixed-32 voting within 1 pp at 2–3$\times$ fewer tokens. In a live 3B-model comparison under a fixed 3,072-token budget, one long response (62%) beats three short voted ones (54%), whose losses come from truncation. Code and results: `https://github.com/alizarman786-prog/atml-pa3`.

1 Introduction

Adapting a pretrained model to a new task and spending more computation at inference are two common ways of improving small models, and both can improve a target metric while quietly changing other behavior. We examine three settings. Task 1 asks how mathematical fine-tuning of Qwen3-0.6B [Qwen Team, 2025b] trades specialization against retention, and how restrictions on adaptation; a starting-point penalty (L2-SP, Li et al., 2018), adapter rank (LoRA, Hu et al., 2022) and orthogonal protection (OPLoRA, Xiong and Xie, 2026), change that balance. Task 2 builds a small vision-language model in the style of LLaVA [Liu et al., 2023] and asks whether its answers depend on the image, using complementary image pairs [Goyal et al., 2017] and image controls. Task 3 asks how much repeated sampling helps when the answer must be selected without a verifier, using self-consistency [Wang et al., 2023], self-certainty [Kang et al., 2025] and VisConf [Usman et al., 2026], sequential refinement [Madaan et al., 2023] and adaptive stopping [Aggarwal et al., 2023]. We report changes in percentage points (pp) with their uncertainty and compute cost.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Table 1: Task 1. Primary scores (%) for the seven evaluation conditions, with the change from the shared baseline in percentage points in parentheses. GSM8K: flexible-extract exact match on 1,319 test questions; HellaSwag and ARC-Easy: normalized accuracy (raw accuracy also shown); BoolQ: accuracy. Largest standard error per benchmark: GSM8K 1.4, HellaSwag 0.5, BoolQ 0.9, ARC-Easy 1.0.

Condition	GSM8K	HellaSwag	BoolQ	ARC-Easy	HS / ARC raw acc	GSM8K trunc. %
Baseline	43.6	47.3	64.4	56.1	37.6 / 60.9	0.8
Full FT (midpoint)	57.0 (+13.4)	45.7 (-1.6)	60.1 (-4.3)	61.7 (+5.6)	36.9 / 65.5	1.0
Full FT	64.7 (+21.2)	46.0 (-1.3)	60.4 (-4.0)	63.5 (+7.4)	37.2 / 66.5	0.5
Full FT + L2-SP	65.0 (+21.5)	46.6 (-0.8)	64.7 (+0.3)	62.3 (+6.3)	37.5 / 65.5	0.8
LoRA r=4	59.7 (+16.1)	47.5 (+0.1)	66.7 (+2.3)	63.0 (+6.9)	37.6 / 65.3	0.9
LoRA r=16	61.8 (+18.2)	47.6 (+0.3)	72.6 (+8.2)	63.9 (+7.9)	37.9 / 66.7	0.8
OPLoRA r=16	61.9 (+18.3)	47.4 (+0.1)	71.6 (+7.2)	63.7 (+7.6)	37.8 / 66.9	0.6

Table 2: Task 1. Trainable parameters, training time, peak allocated GPU memory, size of the saved trainable state and final full-model displacement for the five runs. Method-specific overheads: L2-SP stores a 1.19 GB BF16 copy of θ_0; OPLoRA computes its bases in 37 s and stores 323 MB of them.

Method	Trainable	Time (h)	s/update	Peak mem (GB)	Saved state	$\|\theta - \theta_0\|_2$
Full FT	596.0M	2.53	7.10	12.56	1.20 GB (FP16 model)	10.67
Full FT + L2-SP	596.0M	2.46	6.92	13.75	1.20 GB (FP16 model)	6.27
LoRA r=4	2.5M	2.75	7.86	3.44	10.2 MB adapter	–
LoRA r=16	10.1M	2.80	7.97	3.53	40.5 MB adapter	–
OPLoRA r=16	10.1M	3.66	10.49	3.91	40.5 MB adapter	–

2 Experimental Setup

All training ran on Kaggle with two Tesla T4 GPUs (16 GB each; one run per GPU), PyTorch 2.11.0 and Transformers 5.16.1, with seed 6304 for all data selection, sampling and training. Tasks 1–2 use `Qwen/Qwen3-0.6B` (commit `c1899de`) with thinking disabled. The Task 3 live run uses `Qwen/Qwen2.5-3B-Instruct` [Qwen Team, 2024] at the pinned revision on one Colab T4 (Transformers 5.20.2), because the weekly Kaggle GPU quota had been used. All tables are generated from saved result files by `report/make_report_assets.py`.

3 Task 1: Fine-Tuning Qwen3-0.6B for Mathematics

3.1 Setup and Design Choices

We train on the first 40,000 eligible MetaMathQA [Yu et al., 2024] examples in source order (removing empty examples, those over 2,048 formatted tokens and those whose question matches a GSM8K test question). The loss covers response tokens and the end-of-turn token and is normalized by the number of valid tokens in the effective batch of 32 (micro-batch 2, accumulation 16), for 1,250 AdamW updates with 38 warm-up updates, cosine decay, clipping at 1.0 and FP16 autocast with dynamic loss scaling. Full FT and L2-SP ($\lambda = 10^{-3}$, $\Omega = \frac{1}{2}\|\theta - \theta_0\|^2$ over all unique parameters) use learning rate 2×10^{-5} and FP32 master weights; LoRA (r, α)= $(4, 8)$ and $(16, 32)$ and OPLoRA ($r = 16$, $k = 128$) use 2×10^{-4}, a frozen FP16 backbone and adapters on all seven projections of all 28 layers. OPLoRA projects both factors away from the top-128 singular vectors of each starting weight, computed once in FP32; its constraint holds to 4.3×10^{-5} and merged and unmerged logits agree on 99.7% of next-token argmaxes. We evaluate with lm-evaluation-harness [Gao et al., 2024]: GSM8K [Cobbe et al., 2021] (zero-shot chain of thought, chat template, greedy, 1,024 tokens, flexible extraction) and HellaSwag [Zellers et al., 2019], BoolQ [Clark et al., 2019] and ARC-Easy [Clark et al., 2018] by candidate likelihood. Diagnostics use a probe set of 32 GSM8K and 32 HellaSwag prompts fixed before evaluation.

3.2 Results

Table 1 gives all seven conditions on the four benchmarks, Fig. 1 the training curves, benchmark changes and activation corrections, and Table 2 the cost of each run.

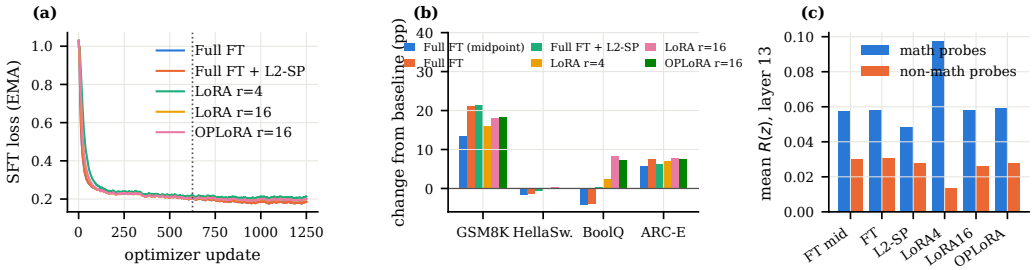


Figure 1: Task 1. (a) SFT loss per update (exponential moving average) for the five runs; dotted line: Full FT midpoint (update 625). (b) Change from baseline (pp) per benchmark. (c) Mean activation correction $R(z) = \|\Delta W z\|/\|W_0 z\|$ of `down_proj` in layer 13 on 32 math and 32 non-math probe prompts; all layers are in Table 9.

3.3 Discussion

RQ1 (specialization and retention). Every method improves mathematics substantially (+16.1 to +21.5 pp on GSM8K, against standard errors of at most 1.4 pp), and the gains are real answers rather than formatting: no condition produces unextractable answers and truncation stays below 1%. Full FT already gains +13.4 pp by its midpoint, two thirds of its final gain, but its BoolQ loss is also already present there (-4.3 at the midpoint, -4.0 at the end), so the cost to reading comprehension is paid early and does not grow with further training. HellaSwag changes by less than 2 pp for every method, and ARC-Easy improves for all of them (+5.6 to +7.9 pp), which suggests that step-by-step math training also helps elementary science multiple choice. Retention is therefore benchmark-specific: only yes/no reading comprehension under full fine-tuning clearly regresses.

RQ2 (restrictions on adaptation). L2-SP gives the clearest improvement in the trade-off. It reaches the same SFT loss as Full FT (0.184 over the last 50 updates for both) and the same GSM8K gain (+21.5 vs. +21.2 pp), yet keeps BoolQ at baseline (+0.3 vs. -4.0 pp) while its final displacement is 6.27 instead of 10.67. The penalty is not negligible: $\lambda\Omega$ reaches about 0.02, roughly a tenth of the SFT loss, so it removes parameter movement that the math objective did not need. Increasing LoRA rank from 4 to 16 lowers the final SFT loss (0.212 to 0.199) and improves both GSM8K (+16.1 to +18.2) and BoolQ (+2.3 to +8.2), so the extra capacity did not cost retention. OPLoRA is indistinguishable from LoRA-16 on every benchmark (all differences ≤ 1 pp, about one standard error). The diagnostics explain why: the fraction of LoRA-16's update lying in the protected top-128 output subspace is 0.12–0.15, close to the 0.125 expected for a direction-blind update of a 1,024-dimensional output, so there was little interference for OPLoRA to remove. Low-rank adapters, like the regularized full model, retain more than unconstrained full fine-tuning [Biderman et al., 2024].

RQ3 (parameter changes and behavior). Update magnitude does not order the methods by retention. The relative update η of `down_proj` is similar for Full FT and LoRA-16/OPLoRA (about 0.01–0.02; Table 9), yet only Full FT loses BoolQ; L2-SP has smaller η than Full FT in all three layers, consistent with its retention. In layers 0 and 13 all corrections respond about twice as strongly to math prompts as to non-math prompts (Fig. 1c), while layer 27 reverses this (ratio 0.66–0.78). LoRA-4 is the most selective (ratios 4.8 and 7.1, and the smallest non-math response) but retains less on BoolQ than LoRA-16, so input selectivity of one matrix does not predict benchmark retention. These measurements isolate local changes on cached inputs: they ignore changes in other layers and how activations shift downstream, and the math probes share one chat-template suffix, which inflates their similarity. They describe where the updates act, not why behavior changed.

RQ4 (performance versus computational cost). Adapters train 0.4–1.7% of the parameters with about a quarter of the peak memory (3.4–3.9 vs. 12.6–13.8 GB) and save a 10–40 MB state instead of a 1.2 GB model, but they are not faster: each update still runs the full network forward and backward, so wall time is 2.75–2.80 h against 2.46–2.53 h for the full models, and OPLoRA's projections add a further 31% (3.66 h). L2-SP costs almost nothing in time but holds a second copy of the model in memory. Overall, L2-SP is the best full-model option and LoRA-16 the best adapter.

Table 3: Task 1. Manual annotation of the 42 fixed responses (first and last underlying questions $\times$ three formats $\times$ seven conditions). Format compliance is judged on the four JSON/template responses per condition, the other columns on all six.

Condition	Format (4 structured) yes/partial/no	Coherence (6) high/med/low	Repetition	Math framing	Inappropriate	Truncated
Baseline	3/0/1	2/1/3	0	0	0	0
Full FT (midpoint)	2/1/1	1/1/4	3	1	0	2
Full FT	2/2/0	1/2/3	2	3	0	1
Full FT + L2-SP	2/2/0	1/4/1	1	3	0	1
LoRA r=4	3/0/1	3/2/1	0	0	0	0
LoRA r=16	2/2/0	2/3/1	0	1	0	0
OPLoRA r=16	2/2/0	2/3/1	1	1	0	0

RQ5 (behavior beyond benchmark scores). The 42 annotated responses (Table 3) show changes the benchmarks miss. The full models attach math-style answer lines to unrelated prompts in three of six responses each, for example ending a wish with "*...I will do that no matter the cost. The answer is: 1*" (Full FT) or prefixing JSON with "*The answer is:*" (L2-SP), which breaks format compliance; LoRA-16 and OPLoRA do this once each and LoRA-4 never. On the plain marital-advice prompt the baseline and all adapters give a short supportive answer, while the Full FT midpoint, final model and L2-SP repeat a paragraph until the 512-token cap. Fine-tuning also helped in one place: the baseline returned the wish template unfilled, whereas every fine-tuned model filled it coherently. L2-SP thus preserves benchmark retention but not conversational behavior, and LoRA-4 behaves most like the baseline. The automatic checks agree (JSON validity 8/8 for the baseline vs. 2–3/8 for fine-tuned models), but the annotation covers only two underlying questions, so these are illustrations rather than estimates.

4 Task 2: SigLIP–Qwen3 Vision-Language Model and Visual Grounding

4.1 Setup and Design Choices

A frozen SigLIP-base encoder [Zhai et al., 2023] produces 196 post-normalization patch features per image, cached once in FP16. A two-layer MLP connector maps them into Qwen3's embedding space, scaled by a constant fixed on training data before training (mean token-embedding norm divided by the initial connector-output norm, 0.055), and the visual tokens are inserted at the start of the user message. Data come from COCO 2014 [Lin et al., 2014] and VQAv2 complementary pairs [Goyal et al., 2017]: 6,000 training images with three captions each; programmatically generated presence pairs (an image with the object covering at least 1% of it, paired with one without) and counting pairs (1–4 non-crowd instances); and natural VQAv2 pairs with $\geq$8/10 annotator agreement and different consensus answers (640/256/640 yes-no/number/other for training). Development and test sets come from val2014, each image used once; the tests contain 64 grounding and 96 VQAv2 pairs. Training continues one model through caption alignment (2,250 updates, connector only), visual instruction tuning (1,500 updates, adding LoRA $r = 16$ on the attention projections; presence/counting/caption replay sampled 50/25/25%) and natural VQA (750 updates; VQA/presence/counting/caption 70/10/10/10%), with an effective batch of 16 and a per-example averaged loss. We evaluate greedy 16-token answers under correct, complementary, mismatched and no-image inputs, scoring grounding by normalized exact match and VQAv2 by the official soft accuracy, with 4,000 bootstrap resamples of complete pairs.

4.2 Results

Fig. 2 shows training and development curves and final accuracy by image condition; Table 4 gives the full final results and Table 5 the correct-image advantages.

4.3 Discussion

RQ1 (learning a visual interface). Caption alignment teaches the model to describe images but not to answer questions from them. Caption development loss falls from 5.42 to 2.43 during Stage 1, and supplying an image lowers answer NLL sharply (the no-image penalty is 4.5 nats on grounding), yet the image-swap penalty stays near 0.1 nats on both suites (Fig. 2b): an image helps, but which image barely matters. Image sensitivity appears as soon as paired questions are trained: within 250 Stage-2 updates the grounding swap penalty rises to 1.8 nats and reaches 4.4 at the end of the stage, and the VQAv2 penalty rises to 1.4 although no VQAv2 question has been trained yet, so the skill transfers across question styles. Stage 3 lowers VQAv2 answer NLL from 2.97 to 1.65 but halves

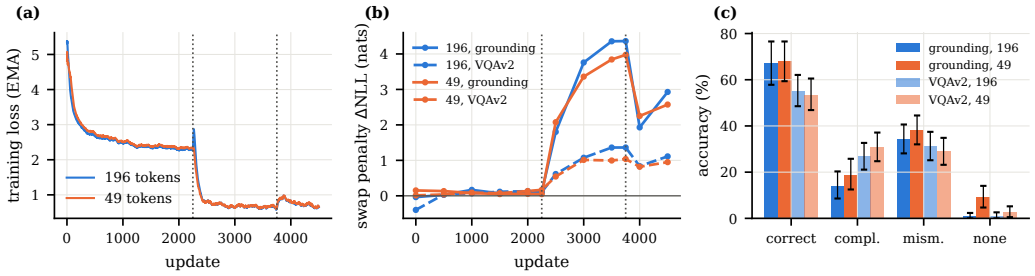


Figure 2: Task 2. (a) Training loss (EMA) for the 196- and 49-token runs; dotted lines mark the stage boundaries at updates 2,250 and 3,750. (b) Development image-swap penalty $\Delta\mathrm{NLL} = \mathrm{NLL}(y \mid q, I_{\mathrm{partner}}) - \mathrm{NLL}(y \mid q, I_{\mathrm{correct}})$ for the grounding and VQAv2 development suites. (c) Final-test accuracy under the four image conditions (95% pair-bootstrap intervals).

Table 4: Task 2. Final-test results of the 196-token model. Accuracy is normalized exact match (grounding) or VQA soft accuracy (VQAv2), with 95% intervals from 4,000 resamples of complete pairs. Pair acc.: both members match their canonical answers; Identical: both members receive the same answer; Follows partner: the prediction equals the partner image's answer; Ref. NLL: mean reference-answer negative log-likelihood (nats).

Image condition	Accuracy [95% CI]	Pair acc.	Identical	Follows partner	Empty	Ref. NLL	ms/answer
Grounding (64 pairs)							
Correct	67.2 [57.8, 76.6]	50.0	20.3	14.1	0.0	1.05	36
Complementary	14.1 [8.6, 20.3]	1.6	20.3	67.2	0.0	3.92	36
Mismatched	34.4 [28.1, 40.6]	3.1	73.4	39.1	0.0	2.34	36
No image	0.8 [0.0, 2.3]	0.0	100.0	0.8	0.0	6.36	114
VQAv2 (96 pairs)							
Correct	55.2 [48.5, 62.1]	28.1	41.7	22.9	0.0	1.48	39
Complementary	26.8 [21.1, 32.7]	3.1	41.7	53.1	0.0	2.66	40
Mismatched	31.2 [25.2, 37.4]	5.2	57.3	32.3	0.0	2.37	43
No image	1.0 [0.0, 2.6]	0.0	100.0	1.0	0.0	9.18	126

Table 5: Task 2. Correct-image advantage over each control in accuracy (pp) and reference NLL (nats), with 95% pair-bootstrap intervals, for the 196-token model and the optional 49-token model.

		vs. complementary		vs. mismatched		vs. no image	
Tokens	Suite	Acc. (pp)	NLL (nats)	Acc. (pp)	NLL (nats)	Acc. (pp)	NLL (nats)
196	Grounding	+53.1 [+39.8, +66.4]	+2.88 [+2.23, +3.54]	+32.8 [+24.2, +41.4]	+1.29 [+0.93, +1.63]	+66.4 [+57.8, +75.8]	+5.31 [+4.55, +6.04]
196	VQAv2	+28.3 [+19.1, +37.9]	+1.17 [+0.82, +1.55]	+24.0 [+15.8, +32.1]	+0.88 [+0.53, +1.25]	+54.1 [+47.2, +61.3]	+7.70 [+6.88, +8.48]
49	Grounding	+49.2 [+35.2, +62.5]	+2.63 [+2.09, +3.14]	+29.7 [+21.1, +37.5]	+1.07 [+0.75, +1.37]	+58.6 [+49.2, +68.0]	+1.93 [+1.52, +2.32]
49	VQAv2	+22.7 [+12.4, +33.4]	+1.09 [+0.77, +1.42]	+24.6 [+16.4, +32.9]	+0.98 [+0.61, +1.37]	+50.9 [+43.9, +58.2]	+5.34 [+4.65, +6.05]

the grounding swap penalty at first (4.4 to 1.9) before it recovers to 2.9, so specialization to natural questions partly erodes the grounding learned in Stage 2 despite 20% grounding replay. Caption loss stays between 2.41 and 2.46 after Stage 1, so caption replay preserved captioning.

RQ2 (grounding and generalization). On annotation-derived questions the model is clearly grounded: accuracy falls from 67.2% with the correct image to 14.1% with the complementary image (+53.1 pp [39.8, 66.4]), and under the swap the prediction equals the partner's answer 67.2% of the time, so it follows the new image rather than producing unsupported answers. Presence is much easier than counting (89.1 vs. 45.3% accuracy; pair accuracy 78.1 vs. 21.9%). Natural questions are less grounded: the advantage over the complementary image halves to +28.3 pp [19.1, 37.9], only 28.1% of pairs are answered correctly for both images, and 41.7% of pairs receive the same answer for both images although their references differ. The residual is concentrated in yes/no questions, which still score 42.6% with the complementary image (a prior near 50%), whereas number and "other" questions drop to 11.5 and 21.3%. Mismatched images give intermediate accuracy (34.4 and 31.2%) because a generic "no" is often right for an unrelated image.

RQ3 (measuring image use). Reference likelihood, generated accuracy and pair metrics agree on the ordering of the image conditions, and every correct-image NLL advantage has an interval well above zero (Table 5). The no-image condition is where they diverge: generated accuracy is near zero (0.8 and 1.0%), but the outputs are full sentences such as "*No, the image does not contain a truck.*" that miss the short-answer format, because the model never saw a question without visual tokens. The NLL penalty measures image use without this format effect. For illustration we take the first three complementary

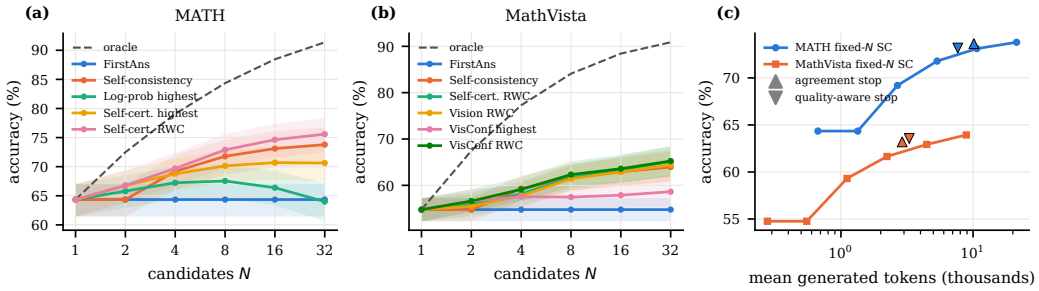


Figure 3: Task 3. (a,b) Held-out accuracy versus candidate budget N for selected rules on MATH and MathVista, with oracle coverage (dashed) and 95% question-bootstrap bands. (c) Accuracy versus mean generated tokens per question for fixed-N self-consistency and the development-selected adaptive stopping policies.

pairs of each outcome category by pair index (`results/task2/report/qualitative.md`). In solved pairs the answer flips with the image ("Does this image contain a truck?": yes with its image, no with the partner's). In half-solved pairs the model gives one answer to both images, "no" to both book images and "yes" to both suitcase images, which is the prior failure that pair accuracy exposes. In failed counting pairs ("How many people can you see?", references 1 and 4) it answers 2 and 3: plausible counts that respond to the image but are not exact.

RQ4 (capabilities and cost). The model answers simple presence questions reliably, counts and answers natural questions only moderately, and relies on priors for yes/no questions. Training updates 1.84M parameters in Stage 1 and 6.42M with LoRA; the 196-token run took 118 min with 6.9 GiB peak memory, plus a one-off 25-minute feature-caching step (Table 12). Averaging 2×2 patch blocks to 49 tokens halves training time (54 min) and memory (3.3 GiB) and cuts generation from 38 to 23 ms per answer, while accuracy stays within the intervals of the 196-token model (68.0 vs. 67.2% grounding, 53.5 vs. 55.2% VQAv2) and the VQAv2 complementary advantage is somewhat smaller (+22.7 vs. +28.3 pp). With 64 and 96 test pairs the intervals span about ±9 pp, so smaller differences between configurations cannot be resolved, and all results come from one seed.

5 Task 3: Test-Time Compute

5.1 Setup and Design Choices

We use the supplied 32 rollouts per question from Qwen2.5-3B-Instruct on 1,000 MATH [Hendrycks et al., 2021] questions and Qwen2.5-VL-3B [Qwen Team, 2025a] on 1,000 MathVista testmini [Lu et al., 2024] questions. Sorted question IDs are permuted with seed 6304; the first 200 form the development split and the remaining 800 the held-out split. Answers are grouped without gold answers using math-verify [Hugging Face, 2025] (MATH) and MathVista's answer-type, choice and precision rules; missing answers form one always-wrong group. We evaluate budgets $N \in \{1, \ldots, 32\}$ as prefixes of ten seeded rollout orders per question, breaking ties by the earliest rollout, and bootstrap questions (1,000 resamples). Quality scores are mean log-probability, self-certainty (mean token KL from uniform), visual engagement and VisConf, whose token confidences are calibrated by the 5th and 95th percentiles of the current candidate set; each is used for highest-score selection and rank-weighted consensus (RWC). Adaptive policies check after 4, 8 and 16 rollouts; thresholds are chosen on the development split as the cheapest setting within 1 pp of fixed-32 self-consistency (SC). The live experiment runs 50 held-out MATH questions in FP16 at batch size 1 with temperature 0.7, top-p 0.8 and top-k 20.

5.2 Results

Fig. 3 shows accuracy against budget and token cost; Table 6 compares all selection rules, Table 7 the live strategies and Table 8 adaptive stopping.

Table 6: Task 3. Held-out accuracy (%, averaged over 10 rollout orders) at N=4 and N=32, and the paired difference from self-consistency at N=32 with 95% intervals from 1,000 question-level bootstrap resamples.

	MATH (800 held-out)			MathVista (800 held-out)		
Method	N=4	N=32	Δ vs SC at 32 [95% CI]	N=4	N=32	Δ vs SC at 32 [95% CI]
Oracle coverage	79.1	91.4	–	77.3	90.9	–
FirstAns	64.4	64.4	-9.4 [-11.1, -7.8]	54.8	54.8	-9.2 [-11.1, -7.2]
Self-consistency	69.2	73.8	–	59.3	63.9	–
Log-prob highest	67.2	64.0	-9.8 [-12.4, -7.2]	54.8	49.4	-14.6 [-17.8, -11.2]
Log-prob RWC	68.5	74.9	+1.1 [+0.1, +2.0]	57.2	62.9	-1.1 [-2.4, +0.3]
Self-cert. highest	68.8	70.6	-3.2 [-5.5, -0.9]	56.3	56.0	-7.9 [-11.2, -4.7]
Self-cert. RWC	69.7	75.6	+1.8 [+0.7, +2.9]	58.1	65.1	+1.2 [-0.3, +2.5]
Shortest (control)	65.2	62.4	-11.4 [-14.1, -8.9]	56.0	54.4	-9.6 [-12.9, -6.5]
Vision highest	–	–	–	55.5	54.5	-9.4 [-12.6, -6.4]
Vision RWC	–	–	–	57.7	64.4	+0.4 [-1.0, +1.9]
VisConf highest	–	–	–	57.5	58.6	-5.3 [-8.2, -2.3]
VisConf RWC	–	–	–	59.1	65.2	+1.3 [-0.1, +2.8]
Token fusion highest	–	–	–	57.1	57.1	-6.8 [-9.8, -3.8]
Token fusion RWC	–	–	–	58.9	64.9	+0.9 [-0.4, +2.3]

Table 7: Task 3. Live comparison on 50 fixed MATH questions with the same 3,072-token output allowance. Tokens and time are per question; refinement includes the shared initial response. Refinement transitions relative to the shared initial answer (initial accuracy 56%): wrong→correct 3, correct→wrong 0, unchanged correct 28, unchanged wrong 19; initial-response truncation 20%, mean critique tokens 582. Parallel oracle (any of 3 correct) 70%. Tesla T4, peak memory 6.17 GiB.

Strategy	Acc. (%)	Input tok.	Output tok.	Time (s)	Truncated (%)	Extracted (%)
Single (1×3,072)	62	108	760	40.5	4.0	96.0
Parallel (3×1,024, SC)	54	325	1857	98.1	16.7	83.3
Refinement (1,024 + 2,048)	62	890	1217	64.8	0.0	100.0

Table 8: Task 3. Adaptive stopping on the held-out split with thresholds selected on the development split. Last column: share of question-orders that stopped early with a wrong answer although fixed-32 self-consistency was correct.

Dataset	Policy	τ	γ	Acc. (%)	Tokens	Rollouts	Stops at 4/8/16/32 (%)	Wrong early, 32 right (%)
MATH	Agreement	0.625	–	73.6	10140	11.8	66/6/1/27	0.64
MATH	Quality-aware	0.5	0.0	73.2	7663	9.0	76/6/1/16	1.64
MATH	Fixed-32 SC	–	–	73.8	21347	32.0	0/0/0/100	0.00
MathVista	Agreement	0.5	–	63.2	2907	8.4	76/8/2/14	4.05
MathVista	Quality-aware	0.5	0.0	63.6	3287	9.6	69/10/3/17	3.21
MathVista	Fixed-32 SC	–	–	63.9	8898	32.0	0/0/0/100	0.00

5.3 Discussion

RQ1 (exploration and selection). Sampling finds correct answers far more often than voting selects them. From 1 to 32 rollouts, oracle coverage rises from 64.4 to 91.4% on MATH and from 54.8 to 90.9% on MathVista, so the share of questions with no correct candidate falls from 35.6 to 8.6% (Table 13). Self-consistency converts only part of this: it gains 9.4 and 9.1 pp and saturates after about eight rollouts (71.8 to 73.8% on MATH from $N = 8$ to 32), while the share of questions where a correct candidate exists but is not chosen grows from 0 to 17.6% (MATH) and 26.9% (MathVista). Additional samples add alternatives (5.9 distinct answers per MATH question at $N = 32$) but mostly repeat the leading answer, so beyond small budgets the bottleneck is selection, and more so on MathVista, where wrong option letters attract consistent votes.

RQ2 (evidence and consensus). Quality scores are weak evidence on their own and useful only in combination with agreement. Highest-score selection falls below self-consistency and gets worse with more candidates (log-probability: 67.5% at $N = 8$, 64.0% at 32 on MATH; 49.4% on MathVista), because a larger pool contains more confidently wrong outliers. Used as vote weights, the same scores help slightly: self-certainty RWC beats self-consistency by +1.8 pp [0.7, 2.9] on MATH, and VisConf RWC by +1.3 pp [−0.1, 2.8] on MathVista, which is not significant. The signal analysis (Table 14) explains the difference between datasets: within a question, rollouts in the top self-certainty quartile are 12.0 pp more often correct than those in the bottom quartile on MATH but only 3.7 pp on MathVista, where VisConf is the strongest signal (+7.2 pp). Log-probability rises with length on MATH (Spearman +0.23) while self-certainty falls (–0.16), and visual engagement is strongly lower

in longer rollouts (–0.69), so part of each score is a length effect; the shortest-rollout control alone is worth +4.6 and +3.6 pp. Confidence and visual engagement are negatively correlated within rollouts (mean –0.21, positive in only 15%), and fusing them per token instead of per rollout changes nothing (64.9 vs. 65.2% with RWC). In the disagreement cases RWC overturns close votes, for example choosing the correct "2" (15 votes) over "0" (16 votes) on one MATH question but preferring a wrong "0" (10 votes) over the correct "4" (11 votes) on another.

RQ3 (breadth and refinement). Under the same 3,072-token allowance, one long response (62%) beats three independent 1,024-token samples (54%) and ties with refinement (62%). The parallel strategy loses to truncation: 16.7% of its samples hit the cap and only 83.3% contain a boxed answer, so when two of three samples end without an answer the missing-answer group wins the vote even if the third sample is correct. Its oracle is 70%, so breadth found answers that voting could not use. Refinement changed three answers from wrong to correct and none from correct to wrong, but all three repaired initial responses had ended without a boxed answer; the review turn finished incomplete solutions rather than correcting mistakes, and 19 wrong answers were kept, for example a minimum of $9/16$ where the answer is -1. Counting actual cost, the single response is also the cheapest (760 output tokens, 40 s) against refinement (1,217 output plus 890 input tokens, 65 s) and parallel sampling (1,857 tokens, 98 s), so for this model and allowance depth is the better use of the budget.

RQ4 (adaptive allocation). Agreement is a good stopping signal. On MATH the agreement policy ($\tau = 0.625$) reaches 73.6% at 10.1k tokens per question against 73.8% for fixed-32 voting at 21.3k, and the quality-aware policy 73.2% at 7.7k; on MathVista the agreement policy reaches 63.2% at 2.9k against 63.9% at 8.9k (Fig. 3c). At similar cost both adaptive policies beat fixed budgets (MATH: 73.1% at $N = 16$ for 10.7k tokens; MathVista: 61.6% at $N = 8$ for 2.2k), because they spend rollouts where they are needed. Stopping tracks difficulty: two thirds of MATH question-orders stop after four rollouts, with 91% accuracy and pools that are 85% correct, while those that run to 32 reach 32% accuracy with pools only 17% correct. The failure mode is consensus on a wrong answer: 32% of MATH and 60% of MathVista question-orders with no correct rollout stop early, and early stops are wrong where full voting would have been right in 0.6% (MATH) and 4.1% (MathVista) of cases, for example a MathVista question stopped at "C" with 2 of 4 votes although 18 of 32 rollouts were correct. The quality-aware rule saved tokens on MATH (7.7k vs. 10.1k) for 0.4 pp less accuracy but cost more on MathVista, so confidence added little beyond agreement.

RQ5 (compute and reliability). The best trade-offs come from voting at moderate budgets with adaptive stopping, which keeps fixed-32 accuracy at a third to a half of its tokens; quality weighting adds at most 2 pp and highest-score selection is unreliable. When the model is consistently wrong, agreement, confidence and visual engagement all point to the wrong answer. Token counts are an offline proxy that omits prefill, attention collection and scoring, and the live results rest on 50 questions, so their differences (8 pp) are within a single question-level standard error of about 7 pp.

6 Conclusion

Across the three tasks, the metric being optimized improved while behavior outside it often did not. Mathematical fine-tuning gave every method a large GSM8K gain, but only regularization or low-rank adaptation avoided a loss in reading comprehension, and none prevented math-style answers or repetition on unrelated prompts. The vision-language model learned to use images only once trained on paired questions, and its natural-question answers remained partly prior-driven, which only pair-level metrics revealed. Repeated sampling produced correct candidates for over 90% of questions, but voting and internal confidence recovered only part of that coverage, and the best savings came from stopping when samples agree. Measurements beyond the target score; pair accuracy, manual inspection, oracle coverage, truncation rates, were needed in each task to see what had actually changed.

Limitations and deviations. All results come from a single seed. The Task 3 live run used Google Colab (one T4, Transformers 5.20.2) instead of Kaggle. MATH answer comparisons that exceeded math-verify's 2 s timeout were counted as non-equivalent. The Task 2 test sets are local subsets (64 and 96 pairs), not server scores. Task 1 likelihood benchmarks were rerun at batch size 8 after an out-of-memory error.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of EMNLP*, 2023.

Dan Biderman et al. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL-HLT*, 2019.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Leo Gao, Jonathan Tow, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024. Version 0.4.13 used.

Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *Proceedings of CVPR*, 2017.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Hugging Face. Math-verify. `https://github.com/huggingface/Math-Verify`, 2025. Version 0.9.0 used.

Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable best-of-n selection for large language models via self-certainty. *arXiv preprint arXiv:2502.18581*, 2025.

Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit inductive bias for transfer learning with convolutional networks. In *International Conference on Machine Learning*, 2018.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of ECCV*, 2014.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations*, 2024.

Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Qwen Team. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025a.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025b.

Usman et al. VisConf: Quality-aware consensus for best-of-N in vision-language model reasoning. *arXiv preprint arXiv:2610.24526*, 2026.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Yifeng Xiong and Xiaohui Xie. OPLoRA: Orthogonal projection LoRA prevents catastrophic forgetting during parameter-efficient fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 34088–34096, 2026.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations*, 2024.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019.

Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of ICCV*, 2023.

A Additional results

Table 9: Task 1. Weight-space and activation diagnostics on `down_proj` (layers 0 / 13 / 27, FP32). ρ_{128}: share of $\|\Delta W\|_F^2$ in the top-128 left singular subspace of W_0 (0.125 for a direction-blind update); $\eta = \|\Delta W\|_F / \|W_0\|_F$; $\bar{R}$: mean $\|\Delta W z\| / \|W_0 z\|$ over 32 math and 32 non-math probes.

Condition	ρ_{128}	η	$\bar{R}$ math	$\bar{R}$ non-math	math / non-math
Full FT (midpoint)	0.134 / 0.149 / 0.116	0.013 / 0.015 / 0.012	0.051 / 0.057 / 0.019	0.025 / 0.030 / 0.029	2.05 / 1.89 / 0.68
Full FT	0.133 / 0.148 / 0.115	0.014 / 0.016 / 0.013	0.053 / 0.058 / 0.019	0.026 / 0.031 / 0.029	2.06 / 1.88 / 0.67
Full FT + L2-SP	0.133 / 0.150 / 0.107	0.011 / 0.014 / 0.008	0.046 / 0.048 / 0.018	0.023 / 0.028 / 0.026	2.00 / 1.74 / 0.70
LoRA r=4	0.133 / 0.139 / 0.105	0.009 / 0.010 / 0.007	0.071 / 0.097 / 0.004	0.015 / 0.014 / 0.005	4.83 / 7.13 / 0.78
LoRA r=16	0.145 / 0.141 / 0.116	0.015 / 0.018 / 0.011	0.045 / 0.058 / 0.010	0.027 / 0.026 / 0.015	1.64 / 2.20 / 0.66
OPLoRA r=16	0.000 / 0.000 / 0.000	0.015 / 0.019 / 0.009	0.048 / 0.059 / 0.009	0.028 / 0.028 / 0.014	1.73 / 2.15 / 0.68

Recovery checkpoint (trainable state + optimizer + scheduler + scaler + RNG): Full FT/L2-SP 7.15 GB; LoRA-4 30.7 MB; LoRA-16/OPLoRA 121.6 MB. Full FT midpoint displacement: 10.03. Skipped (overflow) updates: Full FT 0, Full FT + L2-SP 0, LoRA r=4 0, LoRA r=16 0, OPLoRA r=16 0. Runs were trained two at a time on 2x Tesla T4 (one run per GPU).

Table 10: Task 2. Accuracy / pair accuracy (%) by question type and image condition.

Tokens	Question type	Correct	Complementary	Mismatched	No image
196	counting	45.3 / 21.9	17.2 / 3.1	21.9 / 3.1	1.6 / 0.0
196	presence	89.1 / 78.1	10.9 / 0.0	46.9 / 3.1	0.0 / 0.0
196	number	54.8 / 29.2	11.5 / 0.0	19.2 / 0.0	4.2 / 0.0
196	other	41.1 / 16.7	21.3 / 0.0	17.9 / 5.6	0.0 / 0.0
196	yes/no	69.4 / 38.9	42.6 / 8.3	52.5 / 8.3	0.0 / 0.0
49	counting	51.6 / 28.1	21.9 / 9.4	26.6 / 0.0	10.9 / 0.0
49	presence	84.4 / 68.8	15.6 / 0.0	50.0 / 9.4	7.8 / 0.0
49	number	49.8 / 29.2	15.6 / 0.0	15.6 / 0.0	10.4 / 0.0
49	other	48.5 / 22.2	22.6 / 2.8	15.1 / 2.8	0.0 / 0.0
49	yes/no	61.1 / 27.8	49.3 / 13.9	51.5 / 2.8	0.0 / 0.0

Table 11: Task 2. Final-test results of the optional 49-token model (same columns as Table 4).

Image condition	Accuracy [95% CI]	Pair acc.	Identical	Follows partner	Empty	Ref. NLL	ms/answer
Grounding (64 pairs)							
Correct	68.0 [59.4, 76.6]	48.4	28.1	18.8	0.0	1.02	20
Complementary	18.8 [12.5, 25.8]	4.7	28.1	68.0	0.0	3.64	19
Mismatched	38.3 [32.0, 44.5]	4.7	73.4	37.5	0.0	2.09	19
No image	9.4 [4.7, 14.1]	0.0	100.0	9.4	0.0	2.94	116
VQAv2 (96 pairs)							
Correct	53.5 [46.9, 60.5]	26.0	40.6	27.6	0.0	1.44	25
Complementary	30.9 [24.7, 37.1]	6.2	40.6	51.0	0.0	2.52	24
Mismatched	28.9 [23.2, 34.8]	2.1	55.2	33.3	0.0	2.41	23
No image	2.6 [0.5, 5.2]	0.0	100.0	2.6	0.0	6.78	129

Table 12: Task 2. Training resources by stage. SigLIP feature caching (8,733 train + 528 val images, FP16): 25.3 min wall time in the final Kaggle run (feature extraction step), peak GPU memory 0.44 GiB, 2.79 GB on disk. Greedy generation (correct-image condition, batch 8): 38 ms/answer with 196 tokens, 23 ms/answer with 49 tokens. Calibrated connector scale: 0.0549 (196) and 0.0760 (49).

Tokens	Stage	Updates	Trainable	Time (min)	Peak mem (GiB)
196	Caption alignment	2250	1.84M	55.8	6.83
196	Visual instruction	1500	6.42M	41.9	6.89
196	Natural VQA	750	6.42M	20.5	6.89
49	Caption alignment	2250	1.84M	24.1	3.28
49	Visual instruction	1500	6.42M	19.9	3.33
49	Natural VQA	750	6.42M	9.9	3.33

Further aligned examples (complementary pairs, selection disagreements, premature stops and refinement transitions), each with its selection rule, are in `results/task2/report/qualitative.md` and `results/task3/report/qualitative.md`.

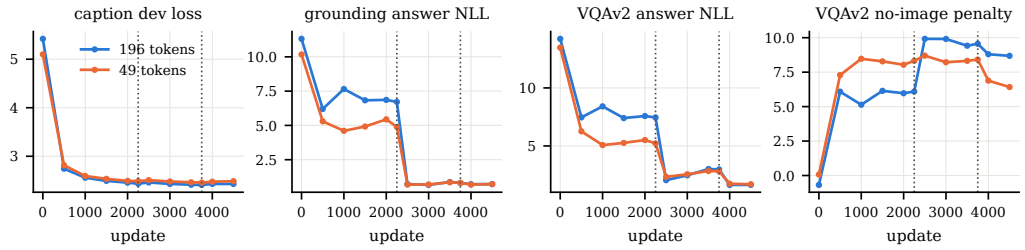


Figure 4: Task 2. Further development metrics; dotted lines mark the stage boundaries.

Table 13: Task 3. Oracle coverage and the split of errors into sampling failures (no correct candidate) and selection failures (a correct candidate was missed), with mean distinct answers and self-consistency token cost (held-out, %). Best RWC: self-certainty RWC (MATH), VisConf RWC (MathVista).

Dataset	N	Oracle	Sampling failure	SC selection failure	Best-RWC selection failure	Distinct answers	Tokens (SC)
MATH	1	64.4	35.6	0.0	0.0	1.00	670
MATH	2	72.5	27.5	8.1	5.7	1.37	1340
MATH	4	79.1	20.9	9.9	9.4	1.93	2680
MATH	8	84.3	15.6	12.6	11.5	2.74	5344
MATH	16	88.4	11.5	15.3	13.8	4.00	10678
MATH	32	91.4	8.6	17.6	15.8	5.91	21347
MathVista	1	54.8	45.2	0.0	0.0	1.00	278
MathVista	2	67.4	32.5	12.7	10.8	1.44	551
MathVista	4	77.3	22.7	18.0	18.1	1.98	1112
MathVista	8	84.1	15.9	22.4	21.8	2.62	2230
MathVista	16	88.4	11.6	25.5	24.9	3.38	4462
MathVista	32	90.9	9.1	26.9	25.6	4.26	8898

Table 14: Task 3. What the scores capture at N=32 (held-out): accuracy difference between each score's top and bottom quartile within a question (averaged), mean within-question Spearman correlation with correctness (questions with mixed outcomes only) and with length. Within-rollout correlation between calibrated confidence $c_{i,t}$ and visual engagement $v_{i,t}$ (MathVista, held-out, 25,572 rollouts): mean -0.211, median -0.208, 10th–90th percentile [-0.440, 0.044], positive in 14.7% of rollouts.

Dataset	Score	Top−bottom quartile acc. (pp)	Spearman with correct	Mixed questions	Spearman with length
MATH	Shortness (−length)	+4.6	+0.076	469	–
MATH	Mean log-prob L_i	+7.8	+0.134	469	+0.225
MATH	Self-certainty K_i	+12.0	+0.198	469	-0.161
MathVista	Shortness (−length)	+3.6	+0.042	643	–
MathVista	Mean log-prob L_i	+2.0	+0.035	643	+0.329
MathVista	Self-certainty K_i	+3.7	+0.050	643	+0.349
MathVista	Visual engagement V_i	+3.2	+0.034	643	-0.686
MathVista	VisConf $\bar{Z}_i$	+7.2	+0.085	643	-0.275
MathVista	Token fusion Z_i^{tok}	+6.9	+0.076	643	-0.322

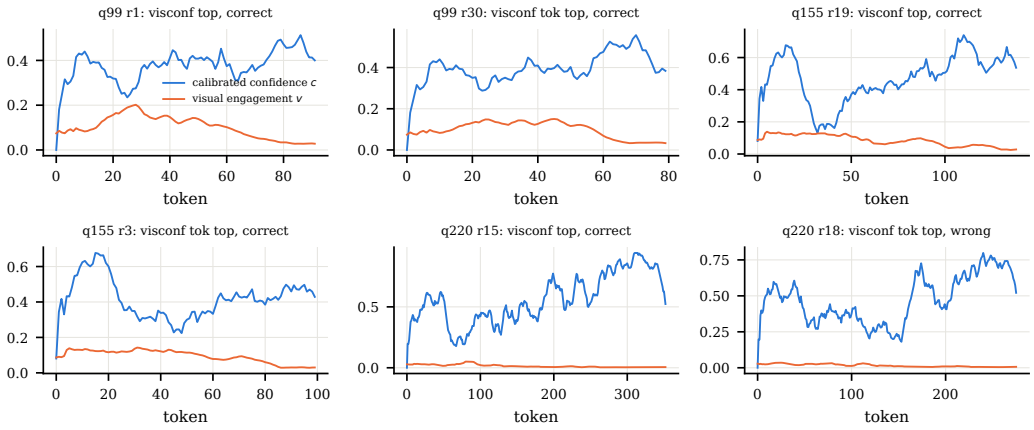


Figure 5: Task 3. Token-level calibrated confidence and visual engagement (15-token moving average) for the fixed illustrative MathVista trajectories: the first held-out questions where VisConf and token fusion select different rollouts at $N{=}32$.